

01 — Reading EXPLAIN Output

"EXPLAIN is the most powerful diagnostic tool in PostgreSQL. Every performance engineer must master it."

Table of Contents

1. [Learning Objectives](#)
 2. [What is EXPLAIN](#)
 3. [EXPLAIN vs EXPLAIN ANALYZE](#)
 4. [Plan Tree Structure](#)
 5. [ASCII Diagram: EXPLAIN Plan Tree](#)
 6. [Node Types Reference](#)
 7. [Cost Fields Explained](#)
 8. [Reading a Real EXPLAIN Output](#)
 9. [EXPLAIN Options](#)
 10. [Understanding Row Width](#)
 11. [Identifying Performance Problems](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Exercises with Solutions](#)
 17. [Production Scenarios](#)
 18. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Run EXPLAIN and EXPLAIN ANALYZE with the correct options
 - Navigate the plan tree hierarchy and understand execution order
 - Interpret cost estimates (startup, total), row estimates, and width
 - Identify the most common node types and what they mean
 - Spot performance red flags in a plan (bad estimates, seq scans, expensive sorts)
 - Use EXPLAIN output to diagnose and fix slow queries
-

What is EXPLAIN

EXPLAIN shows the **query execution plan** chosen by PostgreSQL's optimizer without actually running the query. It displays the sequence of operations (nodes) the planner chose, with estimated costs and row counts.

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
```

Output:

```
Index Scan using idx_orders_customer on orders (cost=0.56..8.58 rows=5 width=128)
  Index Cond: (customer_id = 42)
```

This tells you: the planner will use an Index Scan, estimated cost is 0.56 to 8.58, estimated 5 rows returned, each row approximately 128 bytes wide.

EXPLAIN vs EXPLAIN ANALYZE

Command	Runs Query?	Shows Actual Stats?	Use When
EXPLAIN	No	No	Quick check of plan, safe on production
EXPLAIN ANALYZE	Yes	Yes	Comparing estimates to actuals, diagnosing
EXPLAIN (ANALYZE, BUFFERS)	Yes	Yes + I/O stats	Full diagnosis
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)	Yes	Yes	Machine-readable, for pg_query_plan tools

Important: `EXPLAIN ANALYZE` EXECUTES the query, including any side effects (INSERTs, UPDATEs). For DML:

```
BEGIN;  
EXPLAIN ANALYZE INSERT INTO orders ...;  
ROLLBACK; -- roll back the actual data change
```

Plan Tree Structure

An EXPLAIN plan is a **tree** of nodes. Each node:

- Receives input from its children (0, 1, or 2 children)
- Processes the input
- Returns rows to its parent

The **bottom nodes** read data (Seq Scan, Index Scan). The **top node** produces the final result. Execution is **bottom-up**: leaf nodes run first, feeding data upward.

Indentation = Parent/Child Relationship

Node A	← executes last (top node, produces output)
Node B	← executes second (feeds into A)
Node C	← executes first (feeds into B)
Node D	← executes first (feeds into B)

More indentation = deeper in the tree = executed earlier.

ASCII Diagram: EXPLAIN Plan Tree

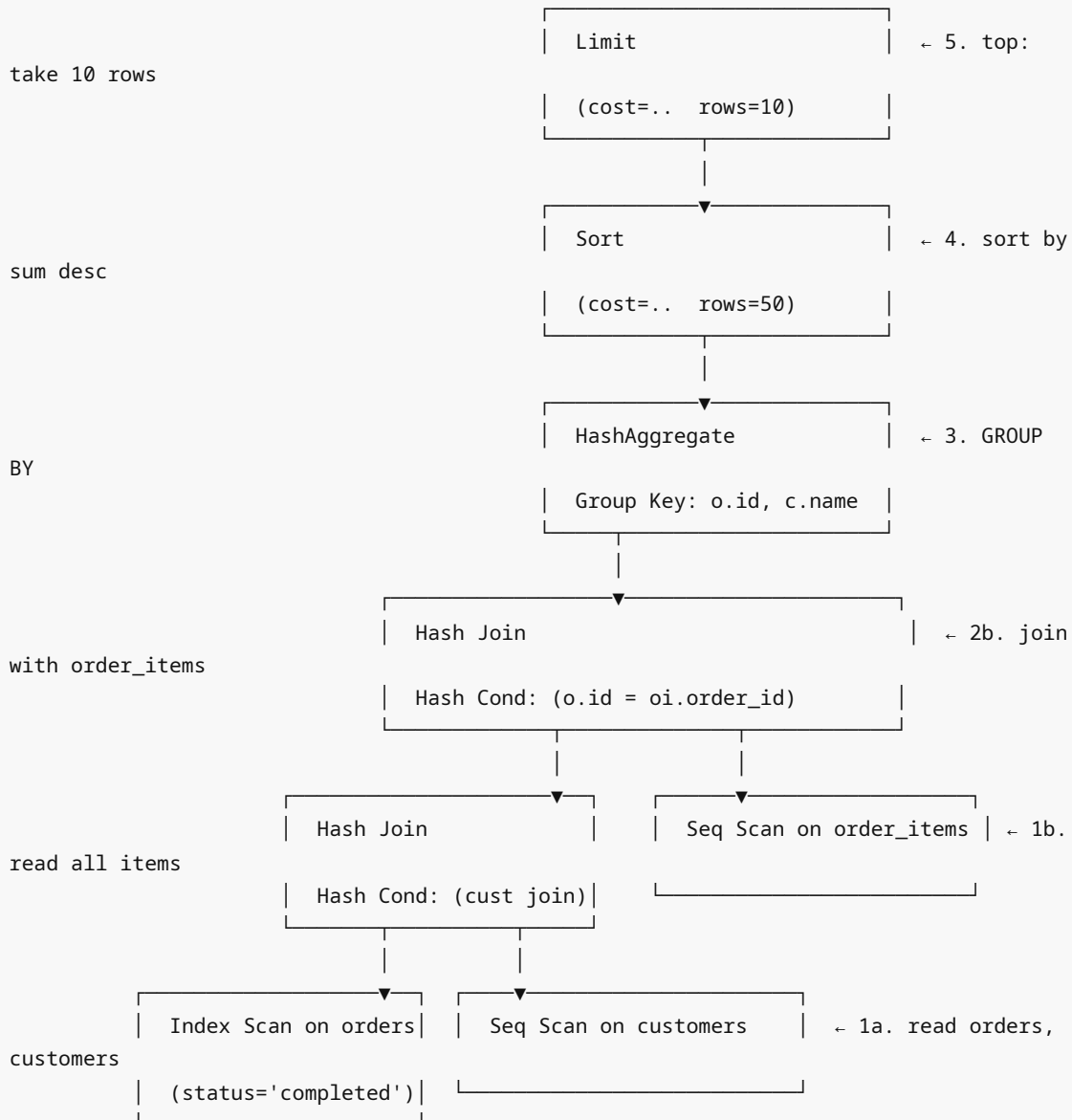
```
EXPLAIN SELECT o.id, c.name, SUM(oi.price)  
FROM orders o  
JOIN customers c ON o.customer_id = c.id
```

```

JOIN order_items oi ON o.id = oi.order_id
WHERE o.status = 'completed'
GROUP BY o.id, c.name
ORDER BY SUM(oi.price) DESC
LIMIT 10;

```

EXPLAIN output (simplified):



Execution order:

- 1a. Read orders (index scan, filter status='completed') AND customers (seq scan)
- 2b. Build hash table from customers, probe with orders (Hash Join)
- ... AND read all order_items (seq scan)
- 2c. Build hash of order_items, join with the orders+customers result
3. GROUP BY o.id, c.name, compute SUM
4. Sort by SUM DESC
5. Take top 10 rows

Node Types Reference

Data Access Nodes (Leaf Nodes)

Node	Description
Seq Scan	Full table scan, reads all pages sequentially
Index Scan	Navigates B-tree, fetches matching heap tuples randomly
Index Only Scan	Uses covering index, no heap fetch
Bitmap Index Scan	Collects ctids into bitmap (feeds Bitmap Heap Scan)
Bitmap Heap Scan	Reads heap pages in bitmap order
TID Scan	Direct lookup by ctid (rare)
CTE Scan	Reads from a WITH clause result

Join Nodes

Node	Description
Nested Loop	For each outer row, scan inner relation
Hash Join	Build hash table from inner, probe with outer
Merge Join	Both inputs sorted on join key, merge

Aggregation/Grouping Nodes

Node	Description
HashAggregate	GROUP BY using hash table (requires work_mem)
GroupAggregate	GROUP BY on sorted input
Aggregate	No GROUP BY, compute overall aggregate
WindowAgg	Window functions

Sort/Limit Nodes

Node	Description
Sort	ORDER BY (uses work_mem; spills to disk if exceeded)
Limit	LIMIT clause
Unique	DISTINCT or UNION (on sorted input)

Other Nodes

Node	Description
------	-------------

Filter	Post-scan predicate evaluation
Result	No base table (e.g., SELECT 1)
Append	UNION ALL, partition scans
Materialize	Cache result for reuse
Gather	Collect results from parallel workers
Gather Merge	Collect sorted results from parallel workers
Memoize	Cache parameterized inner relation (PG14+)

Cost Fields Explained

Every node shows: (cost=startup..total rows=N width=W)

Startup Cost

The cost before returning the **first row**. For a Seq Scan, this is ~0 (can start returning rows immediately). For a Sort node, startup cost is high (must read ALL input before any output).

Total Cost

The cost when the **last row** is returned. This includes all processing.

Cost Units

Costs are abstract units relative to `seq_page_cost = 1.0` :

- `1.0` = cost of reading one sequential 8 KB page
- `random_page_cost` = 4.0 (HDD) or 1.1 (SSD)
- `cpu_tuple_cost` = 0.01
- `cpu_operator_cost` = 0.0025

The absolute values are not important — only relative comparisons between plans matter.

rows

The **planner's estimate** of rows returned by this node. Accuracy depends on table statistics. Grossly wrong estimates (10× off) indicate stale statistics.

width

Average size in bytes of one output row. Used to estimate memory requirements (e.g., how much `work_mem` is needed for a hash table).

Reading a Real EXPLAIN Output

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
```

```

LEFT JOIN orders o ON u.id = o.user_id
WHERE u.created_at > '2024-01-01'
GROUP BY u.id, u.name
ORDER BY order_count DESC
LIMIT 5;

```

```

Limit (cost=2345.67..2345.68 rows=5 width=24)           -- A
  (actual time=45.234..45.235 rows=5 loops=1)
  -> Sort (cost=2345.67..2370.67 rows=10000 width=24)    -- B
        (actual time=45.232..45.233 rows=5 loops=1)
        Sort Key: (count(o.id)) DESC
        Sort Method: top-N heapsort  Memory: 25kB
        -> HashAggregate (cost=1890.00..2015.00 rows=10000 width=24) -- C
              (actual time=40.123..44.567 rows=9876 loops=1)
              Group Key: u.id, u.name
              Batches: 1  Memory Usage: 1825kB
              -> Hash Left Join (cost=567.89..1765.00 rows=50000 width=20) -- D
                    (actual time=5.678..35.123 rows=50123 loops=1)
                    Hash Cond: (u.id = o.user_id)
                    -> Index Scan using idx_users_created on users           -- E
                          (cost=0.43..234.56 rows=8000 width=16)
                          (actual time=0.056..3.456 rows=7890 loops=1)
                          Index Cond: (created_at > '2024-01-01')
                          Buffers: shared hit=234 read=10
                    -> Hash (cost=289.00..289.00 rows=22312 width=8)           -- F
                          (actual time=4.567..4.567 rows=22312 loops=1)
                          Buckets: 32768  Batches: 1  Memory Usage: 1025kB
                          -> Seq Scan on orders                               -- G
                                (cost=0.00..289.00 rows=22312 width=8)
                                (actual time=0.034..2.789 rows=22312 loops=1)
                                Buffers: shared hit=156

Planning Time: 2.345 ms
Execution Time: 45.567 ms

```

Line-by-Line Annotations

Node G (first to execute): Seq Scan on orders

- Reads all 22,312 orders sequentially
- actual rows=22312 matches rows=22312 estimate — statistics are current
- shared hit=156 : all pages in buffer cache (no disk I/O)

Node F: Hash — builds a hash table from orders

- Memory Usage: 1025 KB — needs this much work_mem
- Batches: 1 — fits in memory (no disk spill)

Node E: Index Scan on users using created_at index

- Returns 7,890 rows (estimated 8,000 — good estimate)
- shared hit=234 read=10 — mostly cached, 10 pages from disk

Node D: Hash Left Join — for each user, probe orders hash table

- Hash Cond: `u.id = o.user_id`
- actual rows=50,123 vs estimated 50,000 — excellent estimate

Node C: HashAggregate — GROUP BY `u.id, u.name`

- Memory Usage: 1825 KB, Batches: 1 (fits in `work_mem`)
- Returns 9,876 groups

Node B: Sort — ORDER BY `order_count` DESC

- Sort Method: top-N heapsort Memory: 25kB — only keeps top N in memory (efficient for LIMIT)
- actual rows=5 (only needed top 5)

Node A: Limit — takes first 5 rows

- Startup cost = Total cost (no extra work after Sort provides them)

Key insights:

- Execution time: 45.5 ms total
- No disk spill (`work_mem` sufficient)
- Statistics are accurate (estimates close to actuals)
- The Sort uses top-N heapsort (efficient for LIMIT)

EXPLAIN Options

```
-- Most useful for diagnosis
EXPLAIN (ANALYZE, BUFFERS) query;

-- All options:
EXPLAIN (
    ANALYZE [true|false],    -- run query, show actuals (default: false)
    BUFFERS [true|false],    -- show buffer hit/miss/read/written (default: false)
    TIMING [true|false],     -- show actual time per node (default: true when
ANALYZE)
    SUMMARY [true|false],    -- show planning/execution time totals (default: true
when ANALYZE)
    VERBOSE [true|false],    -- show additional info (output columns, etc.)
    FORMAT [text|json|xml|yaml] -- output format (default: text)
) query;

-- Examples:
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...; -- for tools like pg_query_plan
EXPLAIN (VERBOSE) SELECT ...;                       -- see column names in output
EXPLAIN (ANALYZE, TIMING false) SELECT ...;          -- disable per-node timing
(faster for fast queries)
```

Buffers Output Explanation

```
Buffers: shared hit=234 read=10 dirtied=5 written=2
         temp read=50 written=45
```

```
shared hit=234:    234 pages found in shared_buffers (fast, in-memory)
shared read=10:    10 pages read from OS (disk or OS cache)
shared dirtied=5:  5 shared_buffers pages modified (will need writing)
shared written=2:  2 pages written to disk by this query (rare)
temp read=50:      50 temp file pages read (work_mem exceeded, disk spill!)
temp written=45:   45 temp file pages written (disk spill!)
```

`temp read/written` are RED FLAGS — they indicate `work_mem` is too low for this query.

Understanding Row Width

Row width (in bytes) helps estimate memory needs:

```
Node: Hash  (... rows=50000 width=20)
      Memory: 50000 × 20 bytes = 1 MB

If work_mem < 1 MB: Batches > 1 (spills to disk)
If work_mem >= 1 MB: Batches: 1 (fits in memory)
```

Typical widths:

- Integer/bigint: 4/8 bytes
 - Text: variable (avg length + overhead)
 - Timestamp: 8 bytes
 - JSONB: variable, can be large
-

Identifying Performance Problems

Red Flags in EXPLAIN Output

1. Seq Scan on Large Table

```
Seq Scan on orders  (cost=0.00..45678.00 rows=1000000 width=128)
```

Problem: Reading entire large table. Fix: Add an index for the WHERE predicate.

2. Estimated rows << Actual rows (stale statistics)

```
Seq Scan on orders  (cost=0.00..289.00 rows=5 width=128)
                    (actual time=0.023..456.789 rows=50000 loops=1)
```

Problem: Planner estimated 5 rows but got 50,000. Bad plan choice. Fix: `ANALYZE orders;` or increase `statistics_target`.

3. Sort with disk spill (Batches > 1 or temp files)

```
Sort  (... Sort Method: external merge  Disk: 4096kB)
```

Problem: Sort exceeded `work_mem`, spilled to disk. Fix: `SET work_mem = '100MB';` or restructure query.

4. Nested Loop with large outer table

```
Nested Loop (cost=0.00..9876543.00 rows=1000000 ...)  
-> Seq Scan on orders (rows=1000000)  
-> Index Scan on customers (rows=1) [loops=1000000]
```

Problem: Inner query runs 1 million times. Fix: Usually indicates missing join condition or wrong join type (need Hash Join).

5. Many Rows Removed by Filter

```
Index Scan using idx_orders_status on orders  
Index Cond: (status = 'active')  
Filter: (customer_id = 42)  
Rows Removed by Filter: 99850
```

Problem: Index selected 99,851 rows but only 1 was needed after filter. Fix: Composite index on (customer_id, status) or (status, customer_id).

6. High startup cost for Limit queries

```
Sort (cost=98765.00..98766.00 rows=1 ...)  
Seq Scan ...  
Limit ...
```

Problem: Sort must process ALL rows before Limit can return 1. Fix: Index that provides pre-sorted order.

Common Mistakes

1. Running EXPLAIN without ANALYZE (trusting estimates)

Estimates can be wildly wrong. Always use ANALYZE on slow queries to see actual behavior.

2. Forgetting BUFFERS

Without BUFFERS, you can't tell if slowness is from disk I/O or CPU.

```
-- ALWAYS include BUFFERS for serious diagnosis  
EXPLAIN (ANALYZE, BUFFERS) slow_query;
```

3. Not wrapping DML in BEGIN/ROLLBACK

```
-- WRONG: actually deletes data!  
EXPLAIN ANALYZE DELETE FROM orders WHERE status = 'old';  
  
-- CORRECT:  
BEGIN;  
EXPLAIN ANALYZE DELETE FROM orders WHERE status = 'old';  
ROLLBACK;
```

4. Misreading cost units as milliseconds

Cost units (cost=0.56..8.58) are NOT milliseconds. They are abstract planner units. Only `actual time=0.023..0.031` is in milliseconds (when ANALYZE is used).

5. Looking at only the top node

Performance problems are usually in the bottom nodes (data access). Always trace down to the leaf nodes to understand where time is spent.

Best Practices

1. **Always use (ANALYZE, BUFFERS)** for slow query diagnosis.
2. **Compare estimated vs actual rows** at every node. Discrepancies > 10× indicate statistics issues.
3. **Check total execution time vs planning time** — if planning time is > 10% of execution, query is too complex or statistics are very stale.
4. **Look at temp buffers** — any temp read/write indicates memory spill.
5. **Use FORMAT JSON** with tools like `explain.depesz.com` or `pev2` for visual analysis of complex plans.
6. **Disable specific plan nodes** to test alternatives:

```
SET enable_hashjoin = off;  
EXPLAIN slow_query;  
SET enable_hashjoin = on;
```

Performance Considerations

Planning Time

Complex queries with many tables and indexes take more time to plan. If planning time is comparable to execution time (both under 1ms for simple queries), this is normal. For very complex queries (30+ tables), consider:

```
-- Check if planning is the bottleneck  
EXPLAIN (ANALYZE, SUMMARY) complex_query;  
-- Planning Time: 450 ms    - problem if query runs in 100 ms  
-- Execution Time: 100 ms
```

The Planner's Limits

The PostgreSQL planner uses dynamic programming for join ordering, but for queries with more than `join_collapse_limit` (default 8) tables, it uses a GEQO (genetic algorithm) heuristic which may not find the optimal plan.

Interview Questions & Answers

Q1. What does EXPLAIN show and how does it differ from EXPLAIN ANALYZE?

A: `EXPLAIN` shows the query plan the optimizer chose without executing the query. It displays the tree of operations (nodes) with cost estimates, estimated row counts, and row width. `EXPLAIN ANALYZE` actually executes the query and adds actual timing and row counts to each node, enabling comparison between planner estimates and reality. `EXPLAIN ANALYZE` with `BUFFERS` also shows how many pages were read from `shared_buffers`, OS cache, and disk.

Q2. In EXPLAIN output, what does `cost=0.56..8.58 rows=5 width=48` mean?

A: `cost=startup..total` — 0.56 is the cost before returning the first row (tree navigation), 8.58 is the cost when the last row is returned. These are abstract units relative to `seq_page_cost=1.0`. `rows=5` is the planner's estimate of rows this node will return. `width=48` is the average byte width of one output row (used for memory estimation).

Q3. What are the most important red flags to look for in an EXPLAIN ANALYZE output?

A: Five key red flags:

1. `actual rows >> estimated rows` — stale statistics causing bad plan choice
2. `Sort Method: external merge Disk: XYZ kB` — sort spilled to disk, increase `work_mem`
3. `Seq Scan on large_table` with high row count — missing index
4. `Rows Removed by Filter: N >> actual rows` — index not covering the right columns
5. `temp read=N written=M in Buffers` — hash join or hash aggregate spilled to disk

Q4. How do you determine if a Sort node is using disk or memory?

A: In `EXPLAIN ANALYZE` output, Sort nodes show the sort method:

- `Sort Method: quicksort Memory: NkB` — in-memory, efficient
- `Sort Method: top-N heapsort Memory: NkB` — in-memory, used when `LIMIT` is present
- `Sort Method: external sort Disk: NkB` — spilled to disk (problem!)
- `Sort Method: external merge Disk: NkB` — multi-pass disk merge (very slow!)

If disk is used, increase `work_mem` for the session or query, or restructure to use an index with the correct sort order.

Exercises with Solutions

Exercise 1

Given this `EXPLAIN ANALYZE` output, identify all performance issues:

```
Seq Scan on orders (cost=0.00..289.00 rows=5 width=128)
    (actual time=0.023..456.789 rows=50000 loops=1)
  Filter: (status = 'pending' AND customer_id = 42)
 Rows Removed by Filter: 999950
 Buffers: shared hit=10 read=89000
```

Solution:

- **Issue 1:** Seq Scan on what appears to be a large table (1M rows, ~89K pages read)
 - **Issue 2:** Estimated 5 rows, actually 50,000 → wildly wrong statistics (run ANALYZE)
 - **Issue 3:** 999,950 rows removed by filter → index on (customer_id) or (status, customer_id) would avoid reading all these rows
 - **Issue 4:** read=89000 → 89,000 pages from disk (not cached) → severe I/O
 - **Fix:** ANALYZE orders; and CREATE INDEX ON orders(customer_id, status)
-

Production Scenarios

Scenario 1: Dashboard Query Taking 8 Seconds

A product analytics dashboard was timing out. EXPLAIN revealed:

```
Sort  (cost=987654.00.. Sort Method: external merge  Disk: 102400kB)
  HashAggregate  (Batches: 8  Disk Usage: 45678kB)
    Seq Scan on events  (rows=50000000)
```

Diagnosis:

- HashAggregate spilling to disk (Batches: 8) — work_mem too low
- Sort also spilling to disk
- Seq Scan on 50M rows with no suitable index

Fix:

```
-- Session-level increase for this query
SET work_mem = '256MB';
-- Add index for WHERE predicates
CREATE INDEX CONCURRENTLY ON events(tenant_id, event_date DESC);
-- Materialized view for repeated aggregations
CREATE MATERIALIZED VIEW daily_event_stats AS
SELECT tenant_id, event_date, count(*), sum(value)
FROM events GROUP BY tenant_id, event_date;
CREATE UNIQUE INDEX ON daily_event_stats(tenant_id, event_date);
```

Cross-References

- [02 explain analyze.md](#) — EXPLAIN ANALYZE deep dive with actual vs estimated
- [03 scan types.md](#) — All scan node types in detail
- [04 join algorithms.md](#) — Join node types and algorithms
- [05 query planner.md](#) — How the planner makes decisions
- [../07 Indexes/01 index fundamentals.md](#) — Index fundamentals